

Question 3 - Instruction Set 3 – x295++ - SOLUTION

B. Compiling and assembling a C program using our x295++ instruction set

Table 1

C program (The C code below cannot be modified – it must stay as it is stated below)	x295++ assembly program	x295++ machine code program
$z = (x + y) * (x - y)$	LOAD x, r0 LOAD y, r1 COPY r1, r2 ADD r0, r1 SUB r0, r2 MUL r1, r2 STORE r2, z	1010 000 XXX XXXXXX 0000 <Src1 12 bits - x> 1010 001 XXX XXXXXX 0000 <Src2 12 bits - y> 1001 010 001 XXXXXX 0001 001 000 XXXXXX 0010 010 000 XXXXXX 0011 010 001 XXXXXX 1011 XXX 010 XXXXXX 0000 <Dest 12 bits - z>

C. Evaluating our x295++ instruction set using Memory Traffic criteria

Table 2

x295++ program (1 assembly instruction/ machine code instruction per row)	Fetch (number of memory accesses of word size) + Explain how you obtain your count	Decode/Execute (number of memory accesses of word size) + Explain how you obtain your count
Assembly instruction: LOAD x, r0 Machine code instruction: 1010 000 XXX XXXXXX 0000 <Src1 12 bits - x>	Count: 2 Explanation: fetching 1 instruction that is 2-word wide	Count: 1 Explanation: executing a LOAD requires 1 memory access -> reading value x (16 bits -> 1 word) from memory
Assembly instruction: LOAD y, r1 Machine code instruction: 1010 001 XXX XXXXXX 0000 <Src2 12 bits - y>	Count: 2 Explanation: fetching 1 instruction that is 2-word wide	Count: 1 Explanation: executing a LOAD requires 1 memory access -> reading value y (16 bits -> 1 word) from memory
Assembly instruction: COPY r1, r2 Machine code instruction: 1001 010 001 XXXXXX	Count: 1 Explanation: fetching 1 instruction that is 1-word wide	Count: 0 Explanation: executing an COPY does not require memory access since it is manipulating values held in registers
Assembly instruction: ADD r0, r1 Machine code instruction:	Count: 1 Explanation: fetching 1 instruction that is 1-word wide	Count: 0 Explanation: executing an ADD does not require memory access since it is

0001 001 000 XXXXXX		manipulating values held in registers
Assembly instruction: SUB r0, r2 Machine code instruction: 0010 010 000 XXXXXX	Count: 1 Explanation: fetching 1 instruction that is 1 word wide	Count: 0 Explanation: executing an SUB does not require memory access since it is manipulating values held in registers
Assembly instruction: MUL r1, r2 Machine code instruction: 0011 010 001 XXXXXX	Count: 1 Explanation: fetching 1 instruction that is 1-word wide	Count: 0 Explanation: executing an MUL does not require memory access since it is manipulating values held in registers
Assembly instruction: STORE r2, z Machine code instruction: 1011 XXX 010 XXXXXX 0000 <Dest 12 bits - z>	Count: 2 Explanation: fetching 1 instruction that is 2-word wide	Count: 1 Explanation: executing a STORE requires 1 memory access -> writing value z (16 bits -> 1 word) to memory
Grand Total: 13 memory accesses	Total: 10 memory accesses	Total: 3 memory accesses

Once completed, submit it on Crowdmart as your answer to Question 3.

Challenge Question - Instruction Set 3 – x295++

Would modifying or simplifying our test C program improve the performance of the resulting x295++ executable? Would our modified C program lead to an x295++ executable with fewer instructions? Would our modified C program lead to an x295++ executable performing fewer memory accesses? Yes, it will!

Support your claim (“it will” or “it will not”) by completing the **Table 1 – Challenge** and the **Table 2 – Challenge** found in the downloaded file **A7_Q3_Tables.docx** (or pdf) and reproduced below.

In doing so, make sure that your modified/simplified test C program still produce the same result as the original one.

B. Compiling and assembling a modified/simplified C program using our x295++ instruction set

Table 1 - Challenge

C program Modify/simplify the C code below – must still produce the same result as the original C program listed below	x295++ assembly program	x295++ machine code program
$z = (x + y) * (x - y)$ modified/simplified: $z = x^2 - y^2$	LOAD x, r0 MUL r0, r0 LOAD y, r1 MUL r1, r1 SUB r0, r1 STORE r1, z	1010 000 XXX XXXXXX 0000 <Src1 12 bits - x> 0011 000 000 XXXXXX 1010 001 XXX XXXXXX 0000 <Src2 12 bits - y> 0011 001 001 XXXXXX 0010 001 000 XXXXXX 1011 XXX 001 XXXXXX 0000 <Dest 12 bits - z>

C. Evaluating our x295++ instruction set using Memory Traffic criteria

Table 2 - Challenge

C program Modify/simplify the C code below – must still produce the same result as the original C program listed below	x295++ assembly program	x295++ machine code program
Assembly instruction: LOAD x, r0 Machine code instruction: 1010 000 XXX XXXXXX 0000 <Src1 12 bits - x>	Count: 2 Explanation: fetching 1 instruction that is 2-word wide	Count: 1 Explanation: executing a LOAD requires 1 memory access -> reading value x (16 bits -> 1 word) from memory
Assembly instruction: MUL r0, r0 Machine code instruction: 0011 000 000 XXXXXX	Count: 1 Explanation: fetching 1 instruction that is 1-word wide	Count: 0 Explanation: executing an MUL does not require memory access since it is manipulating values held in registers
Assembly instruction: LOAD y, r1 Machine code instruction: 1010 001 XXX XXXXXX 0000 <Src2 12 bits - y>	Count: 2 Explanation: fetching 1 instruction that is 2-word wide	Count: 1 Explanation: executing a LOAD requires 1 memory access -> reading value x (16 bits -> 1 word) from memory
Assembly instruction: MUL r1, r1 Machine code instruction: 0011 001 001 XXXXXX	Count: 1 Explanation: fetching 1 instruction that is 1-word wide	Count: 0 Explanation: executing an MUL does not require memory access since it is manipulating values held in registers
Assembly instruction: SUB r0, r1	Count: 1	Count: 0

Machine code instruction: 0011 001 001 XXXXXX	Explanation: fetching 1 instruction that is 1 word wide	Explanation: executing an SUB does not require memory access since it is manipulating values held in registers
Assembly instruction: STORE r1, z Machine code instruction: 1011 XXX 001 XXXXXX 0000 <Dest 12 bits - z>	Count: 2 Explanation: fetching 1 instruction that is 2-word wide	Count: 1 Explanation: executing a STORE requires 1 memory access -> writing value z (16 bits -> 1 word) to memory
Grand Total: 12 memory accesses	Total: 9 memory accesses	Total: 3 memory accesses